

Beauty Marketplace - Entrega 4 - Padrões, APIs e IA

Esta entrega consolida o uso de padrões GoF, a documentação formal das APIs e a prova de conceito de Inteligência Artificial do Beauty Marketplace, além de registrar o estágio técnico do projeto no Checkpoint 2.

Objetivo

Apresentar os padrões de projeto aplicados, a documentação completa das APIs do sistema e a forma como a Inteligência Artificial é utilizada na aplicação, sempre conectando esses elementos ao estágio atual do projeto.

1. Identificação do grupo

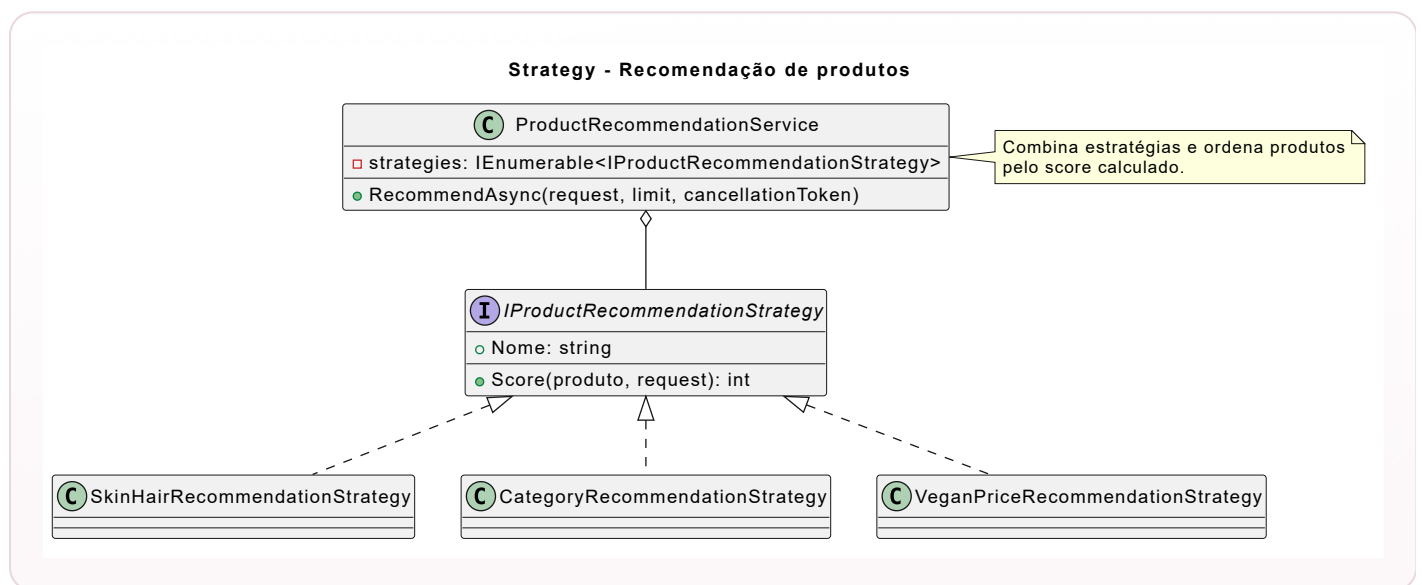
- Número do grupo: **4**
- Integrante 1: **Eduardo Silva de Negreiros** - RA **924109760**
- Integrante 2: **Josué Padetti Correa** - RA **924109806**
- Integrante 3: **Gabrielle Victoria de Souza Barboza** - RA **424106162**
- Integrante 4: **Micael Dantas da Silva** - RA **924110378**
- Integrante 5: **Daiane Jheniffer da Silva Araujo** - RA **1726105657**
- Integrante 6: **Beatriz Cerqueira Sonoro** - RA **924106243**
- Integrante 7: **Kauanne Vitória Soares Bernardo** - RA **924111927**
- Integrante 8: **Júlio Guedes de Oliveira** - RA **926107422**
- Integrante 9: **Matheus Da Silva Reis** - RA **926111266**
- Integrante 10: **Wesley Weber Fernandes** - RA **924107330**
- Integrante 11: **Bryan Willian da Silva Almeida** - RA **925116744**

2. Padrões de projeto (GoF)

O projeto aplica três padrões de projeto distintos para resolver problemas reais do domínio: recomendação flexível, centralização do checkout e seleção do provedor de IA. A seguir, cada padrão é apresentado com categoria, problema resolvido, aplicação no sistema e artefato UML correspondente.

2.1 Strategy - Recomendação de produtos

- **Categoria:** Comportamental.
- **Problema resolvido:** a recomendação precisa combinar critérios diferentes, como tipo de pele, tipo de cabelo, categoria, produto vegano e preço, sem concentrar toda a lógica em um único método rígido.
- **Aplicação no projeto:** `IProductRecommendationStrategy`, `SkinHairRecommendationStrategy`, `CategoryRecommendationStrategy`, `VeganPriceRecommendationStrategy` e `ProductRecommendationService`.
- **Diagrama UML:** `padroes/uml/strategy-recomendacao.puml`.



```

public interface IProductRecommendationStrategy
{
    string Nome { get; }
    int Score(Produto produto, ProductRecommendationRequest request);
}

public sealed class ProductRecommendationService
{
    private readonly IEnumerable<IProductRecommendationStrategy> _strategies;

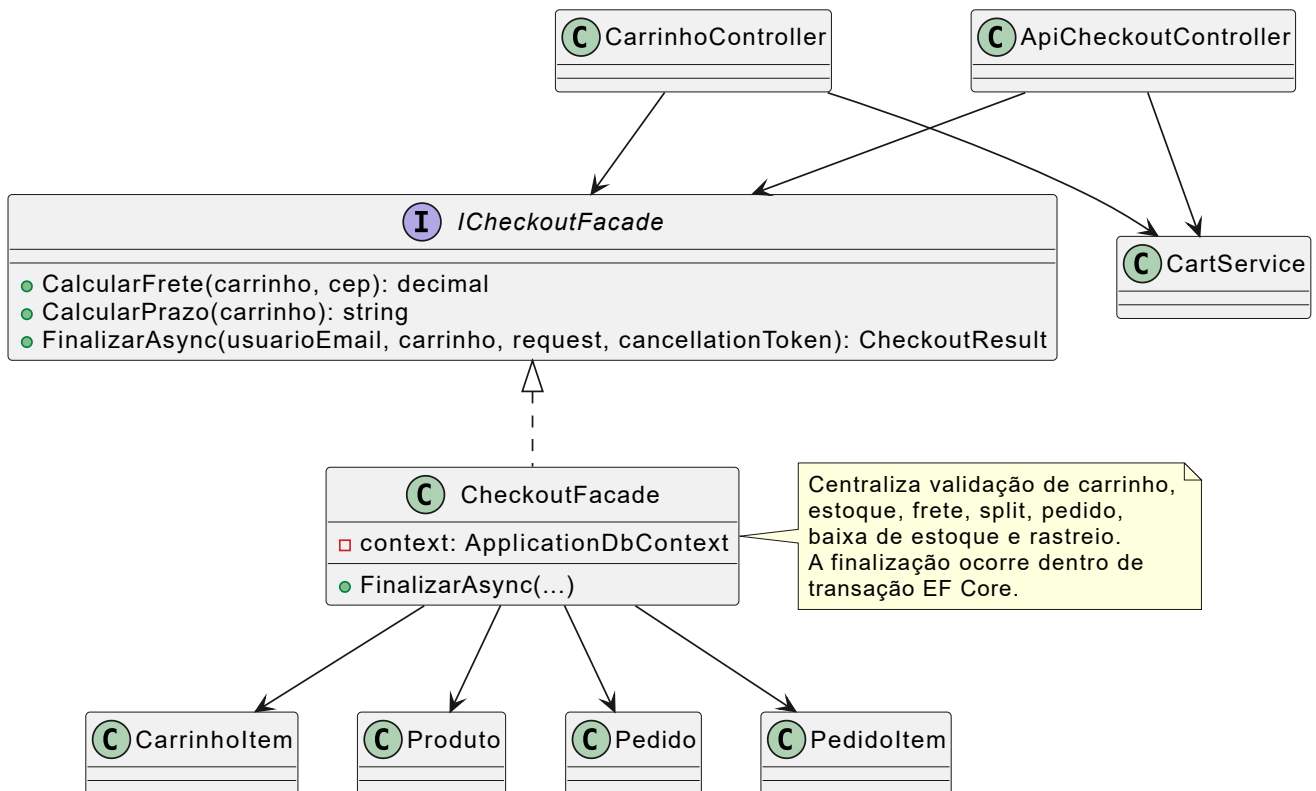
    public Task<IReadOnlyList<Produto>> RecommendAsync(ProductRecommendationRequest request, int
limit, CancellationToken ct)
    {
        // Cada Strategy calcula parte do score de recomendação.
    }
}

```

2.2 Facade - Checkout

- **Categoria:** Estrutural.
- **Problema resolvido:** finalizar uma compra exige validar carrinho, conferir estoque, calcular frete, gerar split, criar pedido, criar itens e baixar estoque. Sem uma fachada, essa sequência ficaria espalhada em múltiplos controllers e serviços.
- **Aplicação no projeto:** `ICheckoutFacade` e `CheckoutFacade`, usados tanto pelo endpoint `POST /api/checkout` quanto pelo checkout MVC.
- **Diagrama UML:** `padroes/uml/facade-checkout.puml`.

Facade - Checkout



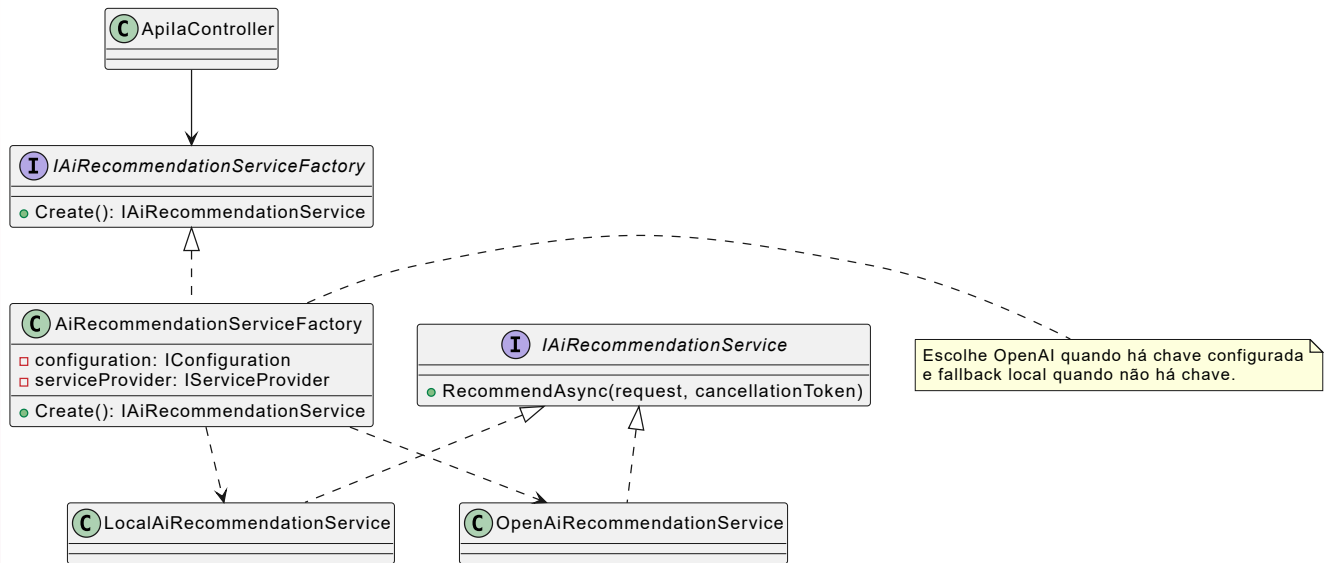
```

public interface ICheckoutFacade
{
    decimal CalcularFrete(IReadOnlyCollection<CarrinhoItem> carrinho, string? cep);
    string CalcularPrazo(IReadOnlyCollection<CarrinhoItem> carrinho);
    Task<CheckoutResult> FinalizarAsync(string usuarioEmail, IReadOnlyCollection<CarrinhoItem> carrinho, CheckoutRequest request, CancellationToken ct);
}
    
```

2.3 Factory Method - Provedor de IA

- **Categoria:** Criacional.
- **Problema resolvido:** o sistema precisa usar OpenAI quando houver chave configurada, mas também deve continuar funcional durante a apresentação acadêmica sem depender de credenciais externas.
- **Aplicação no projeto:** `IAiRecommendationServiceFactory`, `AiRecommendationServiceFactory`, `OpenAiRecommendationService` e `LocalAiRecommendationService`.
- **Diagrama UML:** `padroes/uml/factory-method-ia.puml`.

Factory Method - Provedor de IA



```

public sealed class AiRecommendationServiceFactory : IAIRecommendationServiceFactory
{
    public IAIRecommendationService Create()
    {
        var apiKey = _configuration["OpenAI:ApiKey"] ??
            Environment.GetEnvironmentVariable("OPENAI_API_KEY");
        return string.IsNullOrEmpty(apiKey)
            ? _serviceProvider.GetRequiredService<LocalAiRecommendationService>()
            : _serviceProvider.GetRequiredService<OpenAiRecommendationService>();
    }
}

```

3. Documentação de APIs

O projeto publica sua documentação de APIs em formato OpenAPI/Swagger e também mantém uma coleção Postman versionada no repositório. A documentação foi filtrada para representar apenas as rotas `/api`, evitando misturar endpoints HTTP com ações internas da interface MVC.

Fontes da documentação

- Swagger UI local: `http://localhost:5016/swagger`
- OpenAPI JSON local: `http://localhost:5016/swagger/v1/swagger.json`
- Arquivo versionado: `api/openapi.json`
- Postman Collection: `api/postman_collection.json`

Os endpoints de carrinho, checkout e pedidos exigem autenticação por cookie do ASP.NET Identity com role `Consumidor` . Sem login, retornam `401 Unauthorized` ; com perfil sem permissão, retornam `403 Forbidden` .

Endpoints documentados

Endpoint	Método	Descrição	Status principais
/api/produtos	GET	Lista produtos com filtros de beleza.	200
/api/produtos/{id}	GET	Consulta detalhes de produto.	200, 404
/api/produtos/{id}/avaliacoes	GET	Lista avaliações do produto.	200, 404
/api/produtos/{id}/recomendacoes	GET	Lista produtos recomendados.	200, 404
/api/carrinho	GET	Retorna o carrinho do consumidor.	200, 401
/api/carrinho/itens	POST	Adiciona item ao carrinho.	200, 400, 404
/api/carrinho/itens/{produtoId}	PUT	Atualiza quantidade do item.	200, 404
/api/carrinho/itens/{produtoId}	DELETE	Remove item do carrinho.	200
/api/checkout	POST	Finaliza a compra do carrinho.	201, 400, 401
/api/pedidos	GET	Lista pedidos do consumidor.	200, 401
/api/pedidos/{id}	GET	Consulta pedido com rastreio.	200, 404
/api/ia/recomendacoes	POST	Executa a PoC de recomendação por IA.	200, 400

Total documentado: **12 operações** `/api` , acima do mínimo de 10 endpoints solicitado.

Exemplos da PoC de IA

Request:

```
{
  "tipoPele": "Oleosa",
  "tipoCabelo": "Cacheado",
  "objetivo": "hidratar",
  "vegano": true,
  "precoMax": 120
}
```

Response:

```
{
  "provedor": "Local demonstrativo",
  "resumo": "Recomendação gerada por regras locais que simulam a camada de IA para apresentação sem chave externa.",
  "alertaCompatibilidade": "As sugestões não substituem avaliação dermatológica ou profissional.",
  "produtos": [
    {
      "produtoId": 1,
      "nome": "Base Líquida Maybelline NY Fit Me Matte FPS22",
      "marca": "Maybelline",
      "categoria": "Maquiagem",
      "preco": 51.30,
      "imagemUrl": "/images/produtos/reais/base-liquida-maybelline-ny-fit-me-matte-fps22.jpg",
      "detalhesUrl": "/Produto/Detalhes/1",
      "motivo": "Combina com pele oleosa e acabamento matte."
    }
  ]
}
```

4. Inteligência Artificial na aplicação

A Inteligência Artificial foi planejada para atuar na **recomendação personalizada de produtos de beleza**. O usuário informa seu perfil de beleza e a aplicação retorna sugestões compatíveis com justificativa, imagem, preço e link de detalhes, permitindo apresentar a funcionalidade tanto via API quanto via interface web.

Ferramenta escolhida

- **OpenAI Responses API:** escolhida por permitir geração de texto e respostas estruturadas em uma API atual, adequada para aplicações com IA.
- **Fallback local:** mantém a PoC funcional mesmo sem `OPENAI_API_KEY`, garantindo previsibilidade durante demonstrações.

Referências oficiais:

- [OpenAI Platform](#)
- [Responses API](#)
- [Structured Outputs](#)

PoC implementada

Endpoint: `POST /api/ia/recomendacoes`

Além do endpoint JSON, o projeto possui uma tela de demonstração em **Consumidor > Recomendação IA**, na qual o usuário consegue gerar sugestões visualmente e receber cards com imagem, marca, preço, motivo da recomendação, botão de detalhes e botão de carrinho.

1. O endpoint recebe o perfil de beleza do consumidor.
2. A fábrica escolhe OpenAI quando existe chave configurada.
3. Sem chave, o fallback local usa as estratégias de recomendação do próprio sistema.
4. A resposta retorna produtos aprovados, justificativa, imagem, link de detalhes, provedor utilizado e alerta de compatibilidade.

5. Checkpoint 2 - Estado atual do projeto

Esta seção consolida o estágio atual do sistema e mostra o que já foi entregue tecnicamente no Beauty Marketplace.

Concluído

- Marketplace com separação de perfis: consumidor, lojista e administrador.
- Catálogo com filtros de beleza, slug único, 60 produtos seedados por JSON, imagens reais locais e curadoria de preços compatível com o mercado brasileiro.
- Painel do lojista com cadastro e edição de produtos, incluindo upload validado de imagem.
- Carrinho multi-lojista com persistência por 7 dias, sincronizando sessão e banco local.
- Checkout transacional via Facade, com split, frete, rastreo e baixa de estoque.
- Lista de desejos, avaliações, pedidos e dashboard do lojista.
- Atualização de status de envio pelo próprio lojista.
- Área administrativa para aprovação de lojistas, moderação de produtos e ajuste de comissões.
- Catálogo, API e IA exibindo apenas produtos aprovados.
- Entrega 3 estruturada com C4, SQL, MongoDB e Redis.
- APIs JSON documentadas com Swagger/OpenAPI filtrado para `/api`.

- PoC de IA para recomendação.
- Tela de Recomendação IA disponível para demonstração com perfil consumidor.
- Testes automatizados cobrindo Strategy, Facade/checkout, carrinho persistente, moderação, IA e proteção por perfil.

Links do projeto

- Repositório GitHub: <https://github.com/EduardoSilvaNegreiros/TrabalhoFaculdade5Semestre2026>
- GitHub Projects/Kanban: <https://github.com/users/EduardoSilvaNegreiros/projects/2>
- Nome do Kanban: **Kanban - Projeto 5 Semestre ADS**
- Swagger local: `http://localhost:5016/swagger`
- Postman Collection: `docs/Entrega 4/api/postman_collection.json`

Validação técnica executada

- `dotnet restore` : restauração concluída.
- `dotnet build --no-restore` : compilação concluída com **0 erros e 0 warnings**.
- `dotnet test --no-restore` : **15 testes aprovados**, cobrindo recomendação, checkout, carrinho persistente, roles, catálogo, imagens locais, upload, moderação, IA com cards e proteção por lojista.
- `NuGetAudit=false` : configuração central aplicada para evitar o warning **NU1900**, que dependia de um feed privado externo não autenticado e não estava relacionado ao código da aplicação.

Conferência de commits

Conferência realizada em 31/05/2026. O repositório público e o Kanban foram conferidos. O histórico local visível mostra commits associados a:

- Eduardo <edunegreiros@gmail.com>
- Eduardo Silva de Negreiros <edunegreiros@gmail.com>

6. Critérios de avaliação atendidos

Critério	Peso	Atendimento no projeto
Aplicação e documentação dos padrões GoF	25%	Strategy, Facade e Factory Method foram aplicados, documentados, exemplificados com código e representados em UML.
Qualidade e completude da documentação das APIs	35%	O projeto mantém Swagger/OpenAPI e Postman com 12 operações <code>/api</code> , exemplos e status codes.
Proposta e implementação da funcionalidade de IA	25%	A recomendação por IA foi definida, justificada e implementada com OpenAI + fallback local.
Estado do projeto no Checkpoint 2	15%	O relatório apresenta progresso, validação técnica, GitHub, Kanban e evidências do estágio atual do sistema.